Instruction for BioR Final Test.

Choose 1 dataset

- Pick **one** dataset from the `BioR_test_data/` folder.
- If you prefer, you may use a dataset of your choice, but **all requirements must still be satisfied**. If you use your own data, make sure you have your supervisor's approval and that you are allowed to share and analyse the dataset for this test.

What you must deliver

- **One commented R script** that runs **top-to-bottom**.
- An `outputs/` folder containing exported tables and/or figures, compressed as a **.zip** file.
- A short **PowerPoint** to present your workflow and results.

Script requirements (at least 1 programming element and at least 4 analysis elements + comments)

Comments must explain **what and why** (not only what the code does), especially for:

1. **QC choices** (flags, removal/aggregation decisions)
2. **Joins** (row counts before/after + interpretation)
3. **Programming elements** (custom function / if–else / for loop)

Core programming requirement (must be visible in the script)

Your script must include **at least one** of the following programming elements:

1. **One custom function** you wrote (e.g., `se_fun()`, `iqr_flag()`, `std_id()`, `eu_distance()`), used at least once on real data.
2. **One if/else decision** that changes behaviour based on the data or user choice (e.g., "if too many NAs, drop; else keep", "if duplicates exist, average them; else continue").
3. **One for loop** used for a real task (e.g., loop over sites/genes/populations to compute summaries/plots and save outputs).
   (You may use `purrr::map()` as an advanced alternative, but please include at least one actual `for` loop.)

Presentation (15 minutes)

You will give a short presentation of your analysis using your PowerPoint and your commented R script (sent by email in advance).

Timing

- ~10 minutes: present the workflow and key results (guided by your script)
- ~5 minutes: questions / discussion

Required slides (minimum)

1. **Dataset overview** *(mandatory if you use your own data)*

- What is the dataset about (biological context)?
- What are the main variables and units?
- What is the sampling design/structure (groups, time, replicates)?
- What is the main question you answered?

2. **Programming in the workflow** *(mandatory for everyone)*
   Briefly show and explain:
   - the custom function you wrote (what it does, where it is used)
   - the if/else decision (what condition you checked, what action you took)
   - the for loop output (what you looped over and what it produced)
3. **Results**
   - Show your key plot(s) and/or summary table(s)
   - One clear take-home message + 1 limitation + 1 possible next step

Submission instructions

- Send your commented R script in advance (at least 4 days before) to:
  antonio.profico@unipi.it
  luca.rindi@unipi.it
- Include an `outputs/` folder containing exported tables and figures, compressed as a **.zip** file.
- Name your script: **Surname_Name_BioRtest.R**
  Name the zip file using the same convention: **Surname_Name_BioRtest.zip**

Below a detailed description of the required analysis and programming elements that the script must contain

Required analysis steps (same for all datasets)

Choose **at least 4** of the following analysis components (please consider that 1 and 7 are mandatory):

1. **Import**
   - Use `setwd()` **or** an RStudio Project
   - Use `here()`/relative paths if you want (optional)
   - Read at least one file (`read_csv()`, `read.csv()`, `read_excel()`, etc.)
   - Keep imports clearly labeled (objects named logically)

2. **Inspect**

   - Use `glimpse()` + `skim()` (or base `summary()`)
   - Show `nrow()`, `ncol()`, and key variable types (briefly)

3. **Tidy / reshape**

   - Do **at least one** reshape step: `pivot_longer()` or `pivot_wider()`

4. **Parse / clean IDs**

- Extract information from IDs using **one** of: `separate()`, `extract()`, `parse_number()`, `str_replace()`, `str_split()`

5. **Quality control (≥2 QC flags)**

   - Create ≥2 logical QC columns (e.g., row withd values larger or lower a given threshold)
   - Explain what you did with flagged values (keep/drop/replace/aggregate)

6. **Join tables (or justify no join)**

   - If multiple tables exist, perform ≥1 join (`left_join()`, `inner_join()`, etc.)
   - Report row counts before/after and interpret changes

7. **Summaries + visualisation + export**

   - Produce:
     - **one plot with raw data visible**
     - **one plot with summary + uncertainty**
   - Export to `outputs/`:
     - `clean_dataset.csv`, `summary_table.csv`, `fig1.png`, `fig2.png`

Programming tasks

- **Function:** write `se_fun(x)` or `iqr_flag(x)` and use it in your pipeline.
- **If statement:** implement a QC decision rule, e.g.
  - if duplicated keys exist → average duplicates and add `qc_duplicate = TRUE`
  - else set `qc_duplicate = FALSE` and continue
- **For loop:** loop over groups (site / genotype / population) and save outputs:
  - `outputs/plot_<group>.png` (at least 3 plots)
  - optionally `outputs/summary_<group>.csv`

Example "control flow" patterns

Custom function (example: SE)

```
se_fun <- function(x) {
  x <- x[!is.na(x)]
  sd(x) / sqrt(length(x))
}
```

If/else example (duplicate strategy)

```
dup_n <- sum(duplicated(dat$key))
if (dup_n > 0) {
  message("Duplicates found: ", dup_n, " → averaging duplicates")
  dat <- dat %>%
    group_by(key) %>%
    summarise(value = mean(value, na.rm = TRUE),
              qc_duplicate = TRUE,
              .groups = "drop")
} else {
  dat <- dat %>% mutate(qc_duplicate = FALSE)
}
```

For loop example (save one plot per site)

```
sites <- sort(unique(dat$site))
for (s in sites) {
  p <- ggplot(filter(dat, site == s), aes(x, y)) + geom_point()
  ggsave(filename = paste0("outputs/plot_", s, ".png"),
         plot = p, width = 6, height = 4)
}
```